

DEEP LEARNING – IAT-1

All 10-Mark Questions – Exam-Ready / Handwritten-Style Answers

MMC321 | Semester II | Based on the uploaded question paper

Q1. Backpropagation in Deep Neural Networks with Cross-Entropy Loss

Introduction

Backpropagation is an algorithm used to train multilayer neural networks. It calculates the gradient of the loss with respect to each weight by propagating the error from the output layer towards the input layer using the chain rule.

Forward propagation

For layer l , the weighted input is:

$$z(l) = W(l)a(l-1) + b(l)$$

and activation is:

$$a(l) = f(z(l)).$$

The input is passed through all layers to obtain the prediction $\hat{y}$.

Cross-entropy loss

For a multi-class classification problem:

$$L = - \sum y_i \log(\hat{y}_i)$$

where y is the actual class vector and $\hat{y}$ is the predicted probability vector.

Output error

When softmax is combined with cross-entropy, the output-layer error simplifies to:

$$\delta(L) = \hat{y} - y.$$

Hidden-layer error

Using the chain rule:

$$\delta(l) = (W(l+1)^T \delta(l+1)) \odot f'(z(l))$$

where $\odot$ represents element-wise multiplication.

Weight and bias gradients

The weight gradient is:

$$dL/dW(l) = \delta(l)(a(l-1))^T$$

and the bias gradient is:

$$dL/db(l) = \delta(l).$$

Weight update

Using learning rate η :

$$W(l)_{\text{new}} = W(l)_{\text{old}} - \eta [dL/dW(l)]$$

Therefore:

$$W(l)_{\text{new}} = W(l)_{\text{old}} - \eta \delta(l)(a(l-1))^T.$$

Similarly:

$$b(l)_{\text{new}} = b(l)_{\text{old}} - \eta \delta(l).$$

Working steps

1. Initialize weights and biases. 2. Perform forward propagation. 3. Calculate prediction and cross-entropy loss. 4. Calculate output error. 5. Propagate errors backward. 6. Calculate gradients. 7. Update weights and biases. 8. Repeat for batches and epochs.

Diagram

Input → Forward Propagation → Prediction → Cross-Entropy Loss → Backpropagation → Gradients → Weight Update → Next Iteration

Conclusion

Backpropagation efficiently trains multilayer networks by calculating gradients and updating parameters to reduce loss. The combination of softmax and cross-entropy gives the simple output gradient $\hat{y} - y$.

Q2. Compare CNN and RNN

Introduction

CNNs and RNNs are deep-learning architectures designed for different forms of structured data. CNNs mainly learn spatial patterns, while RNNs learn sequential and temporal relationships.

CNN architecture

A CNN generally contains convolution layers, activation functions, optional pooling layers and a final classifier. Convolution filters scan across the input and learn local features such as edges, textures and shapes.

RNN architecture

An RNN processes an input sequence one time step at a time. It maintains a hidden state that carries information from previous time steps.

Weight sharing

CNNs share the same filter weights across different spatial locations. RNNs share recurrent weights across different time steps. Thus both reduce the number of independent parameters, but they share them in different dimensions.

Spatial invariance

CNNs provide translation-related robustness because the same filter is applied at many spatial positions. Pooling can further reduce sensitivity to small positional changes. RNNs are not primarily designed for spatial invariance.

Applications

CNN: image classification, object detection, face recognition and medical image analysis. RNN: speech sequences, time-series prediction, language modelling and sequential classification.

Comparison

CNN — spatial data, convolution, shared spatial filters, local patterns. RNN — sequential data, recurrent hidden state, shared temporal weights, time dependencies.

Advantages

CNNs are efficient for spatial feature extraction. RNNs can process variable-length sequences and model temporal dependencies.

Limitations

Basic RNNs may suffer from vanishing or exploding gradients over long sequences. CNNs do not naturally maintain a recurrent memory state.

Conclusion

CNNs are suitable for spatial patterns such as images, whereas RNNs are suitable for ordered sequences and temporal relationships.

Q3. 7×7 Input, 3×3 Filter, Stride 1 and Padding 1

Given

Input $N = 7 \times 7$, filter $F = 3 \times 3$, stride $S = 1$, padding $P = 1$.

Output formula

Output size = $((N - F + 2P) / S) + 1$.

Calculation

Output = $((7 - 3 + 2(1))/1) + 1 = 6 + 1 = 7$.

Therefore, output feature map = 7×7 .

Reason

Padding of one cell around the input compensates for the reduction caused by a 3×3 filter. With stride one, the spatial size is preserved.

Trainable parameters

For one 3×3 filter and one input channel, weights = $3 \times 3 = 9$. If a bias is used, one more parameter is added. Therefore total = $9 + 1 = 10$.

Important note

The question does not specify input channels or number of filters. Under the natural single-channel, single-filter interpretation, the answer is 10 parameters including bias. Without bias it is 9.

Conclusion

The convolution produces a 7×7 feature map and a single 3×3 filter contains 10 trainable parameters when one bias is included.

Q4. Vanishing and Exploding Gradients; ReLU and ResNet

Introduction

In a deep network, backpropagation repeatedly multiplies derivatives. Very small products cause vanishing gradients, while very large products cause exploding gradients.

Vanishing gradient

When gradients become close to zero, early layers receive extremely small updates. They learn very slowly and may fail to learn useful representations.

Exploding gradient

When gradients become very large, weight updates become unstable. The loss may oscillate or training may diverge.

Causes

Repeated multiplication of derivatives smaller than one can shrink gradients. Large derivatives can make them grow. Saturating activations such as sigmoid and tanh can also contribute to vanishing gradients.

ReLU

$\text{ReLU}(x) = \max(0, x)$. For positive x , its derivative is 1, so positive activations can pass gradients without the same saturation effect seen in sigmoid or tanh.

ReLU limitation

For negative x , the derivative is zero. A neuron that remains negative can stop receiving useful updates, called the dying-ReLU problem.

Residual connections

A ResNet block uses a shortcut connection:

$$y = F(x) + x.$$

The shortcut creates a direct path for information and gradients, making very deep networks easier to optimize.

Comparison

ReLU improves gradient flow for positive activations. Residual connections provide direct paths across layers. Together with proper initialization, normalization and optimization, they help train deep networks.

Diagram

Input $x \rightarrow [\text{Convolution/Activation Layers } F(x)] \rightarrow \text{Add } x \rightarrow \text{Output } y$

Conclusion

Vanishing gradients slow learning and exploding gradients destabilize learning. ReLU and residual connections are important techniques for improving gradient propagation in deep networks.

Q5. LSTM Architecture and Gates

Introduction

Long Short-Term Memory (LSTM) is a type of recurrent neural network designed to learn long-term dependencies. It uses a cell state and gates to control information flow.

Main components

An LSTM receives $x(t)$, previous hidden state $h(t-1)$, and previous cell state $C(t-1)$. It contains a forget gate, input gate, candidate state and output gate.

Forget gate

The forget gate decides what information should be removed:

$$f(t) = \text{sigmoid}(W_f[h(t-1), x(t)] + b_f).$$

Values near 0 forget information and values near 1 retain it.

Input gate

The input gate controls how much new information enters:

$$i(t) = \text{sigmoid}(W_i[h(t-1), x(t)] + b_i).$$

Candidate state

New candidate information is generated as:

$$\tilde{C}(t) = \tanh(W_C[h(t-1), x(t)] + b_C).$$

Cell state

The cell state is updated by combining retained old information and selected new information:

$$C(t) = f(t) \odot C(t-1) + i(t) \odot \tilde{C}(t).$$

Output gate

The output gate controls information exposed as the hidden state:

$$o(t) = \text{sigmoid}(W_o[h(t-1), x(t)] + b_o)$$

$$h(t) = o(t) \odot \tanh(C(t)).$$

Roles

Forget gate = removes unnecessary information. Input gate = selects new information. Cell state = carries long-term information. Output gate = controls the current hidden output.

Advantages

LSTM handles long-term dependencies better than a basic RNN and is useful in language, speech and time-series tasks.

Conclusion

The combination of gates and cell state gives LSTM controlled memory and helps it learn long-term relationships.

Q6. Self-Attention and Multi-Head Attention

Introduction

Self-attention allows each token to examine other tokens in the same sequence and assign different importance to them. It is a core mechanism of Transformer networks.

Q, K and V

For input X:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

Query represents what a token is looking for, Key is used for matching, and Value contains information to be aggregated.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V.$$

First QK^T produces similarity scores. Division by $\sqrt{d_k}$ scales the values. Softmax converts them into weights. The weighted values are combined.

Self-attention

It is called self-attention because Q, K and V are generated from the same input sequence. Thus each token can directly use information from other tokens.

Multi-head attention

Multi-head attention performs several attention operations in parallel using different learned projections.

Equation

$$\text{head}_i = \text{Attention}(QW_iQ, KW_iK, VW_iV)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O.$$

Difference

Scaled dot-product attention is one attention operation. Multi-head attention uses multiple scaled dot-product attention heads and combines their outputs.

Advantages

Self-attention captures long-range relationships and allows parallel processing. Multi-head attention can learn different types of relationships at the same time.

Applications

Transformers are used in language modelling, translation, summarization, question answering and many other sequence-learning applications.

Conclusion

Scaled dot-product attention is the basic operation, while multi-head attention is a parallel collection of attention operations.

Q7. Overfitting, Dropout, L2 Regularization and Early Stopping

Introduction

Overfitting occurs when a model learns the training data too closely, including noise or accidental patterns, and therefore performs poorly on unseen data.

Symptoms

Training accuracy may be very high while validation/test accuracy is much lower. Training loss may continue decreasing while validation loss starts increasing.

Causes

Large model complexity, insufficient training data, noisy data, too many parameters and excessive training can cause overfitting.

Dropout

Dropout randomly deactivates a fraction of neurons during training. It reduces dependence on particular neurons and encourages robust representations.

L2 regularization

L2 adds a penalty for large weights:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum w^2.$$

This discourages excessively large weights and reduces model complexity.

Early stopping

Early stopping monitors validation performance and stops training when it no longer improves for a chosen patience period. The best validation checkpoint can be retained.

Evaluation

Dropout reduces co-adaptation. L2 controls weight magnitude. Early stopping prevents unnecessary training. They may be used separately or together.

Limitations

Too much dropout can cause underfitting. Excessive L2 can restrict learning. Early stopping requires a suitable validation set and patience value.

Conclusion

Overfitting is a generalization problem. Dropout, L2 regularization and early stopping are important techniques for improving performance on unseen data.

Q8. Batch Normalization During Training and Inference

Introduction

Batch Normalization normalizes intermediate activations using statistics from a mini-batch during training. It also learns scale and shift parameters.

Training mean

For a batch of m values:

$$\mu_B = (1/m) \sum x_i.$$

Training variance

Variance is:

$$\sigma_B^2 = (1/m) \sum (x_i - \mu_B)^2.$$

Normalization

The normalized value is:

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}.$$

Scale and shift

The normalized activation is transformed using learned parameters γ and β :

$$y_i = \gamma \hat{x}_i + \beta.$$

Inference

During inference, current test-batch statistics are normally not used. Running estimates of mean and variance accumulated during training are used instead.

Effect

Batch Normalization can make optimization more stable and may allow effective use of larger learning rates. It can also provide a regularizing effect in some situations.

Internal covariate shift

Batch Normalization was originally motivated partly by reducing changes in intermediate activation distributions. Modern discussions also emphasize its optimization and conditioning benefits.

Limitations

Very small or highly variable batches can reduce its usefulness. Other normalization methods may be preferable in some architectures.

Conclusion

During training Batch Normalization uses batch statistics, then scales and shifts activations. During inference it uses stored running statistics.

Q9. Transfer Learning using ResNet50/VGG16; Feature Extraction vs Fine-Tuning

Introduction

Transfer learning reuses knowledge learned by a model on a large dataset for a new target task. Pretrained CNNs such as ResNet50 and VGG16 can provide useful visual features.

Step 1

Load the pretrained architecture and its learned weights.

Step 2

Remove or replace the original classification head and add a new classifier for the custom classes.

Feature extraction

Freeze the pretrained convolutional layers and train only the new classifier. The pretrained network acts as a fixed feature extractor.

Fine-tuning

After initial training, selected pretrained layers are unfrozen and trained with a small learning rate along with the new classifier. This adapts the features to the target dataset.

Workflow

Dataset → preprocessing → pretrained CNN → replace classifier → freeze backbone → train classifier → optionally unfreeze selected layers → fine-tune → evaluate.

Comparison

Feature extraction: frozen backbone, faster, lower computation, useful when target data is limited or similar to the original domain.

Fine-tuning: some layers updated, more adaptable, more computation and greater risk of overfitting.

Advantages

Transfer learning reduces training time, works well with limited data and reuses useful representations learned from large datasets.

Precautions

Use correct preprocessing, separate training/validation/test data, use a small learning rate during fine-tuning and monitor overfitting.

Conclusion

Feature extraction is a good starting point, while fine-tuning gives greater task-specific adaptation when enough target data and computation are available.

Q10. SGD with Momentum vs RMSprop vs Adam

Introduction

Optimization algorithms update model parameters to minimize the loss function. SGD with Momentum, RMSprop and Adam differ in how they use current and previous gradients.

SGD with Momentum

Momentum accumulates a moving direction of gradients:

$$v(t) = \beta v(t-1) + g(t)$$

$$\theta(t) = \theta(t-1) - \eta v(t).$$

It accelerates movement in consistent directions and reduces oscillations.

RMSprop

RMSprop maintains an exponential average of squared gradients:

$$s(t) = \beta s(t-1) + (1-\beta)g(t)^2$$

$$\theta(t) = \theta(t-1) - \eta g(t)/(\sqrt{s(t)} + \epsilon).$$

Thus the effective learning rate adapts to gradient magnitude.

Adam

Adam combines first- and second-moment information:

$$m(t) = \beta_1 m(t-1) + (1-\beta_1)g(t)$$

$$v(t) = \beta_2 v(t-1) + (1-\beta_2)g(t)^2.$$

Bias-corrected estimates are then used for the parameter update.

Adaptive learning rates

SGD with Momentum normally uses a base learning rate with momentum. RMSprop adapts the effective step size using squared-gradient history. Adam combines adaptive scaling with momentum-like first-moment information.

Convergence

SGD with Momentum can give strong generalization but may need careful learning-rate tuning. RMSprop can work well when gradient scales vary. Adam often reaches useful solutions quickly because it combines adaptive scaling and momentum.

Comparison

SGD + Momentum: momentum-based update, generally non-adaptive base learning rate. RMSprop: adaptive per-parameter scaling from squared gradients. Adam: adaptive scaling plus first- and second-moment estimates.

Advantages

Momentum reduces oscillation. RMSprop adapts step sizes. Adam is convenient and often converges quickly with comparatively little manual tuning.

Limitations

All optimizers require suitable hyperparameters. Fast Adam convergence does not automatically mean best generalization, while SGD may require more tuning.

Conclusion

SGD with Momentum focuses on accumulated direction, RMSprop adapts using squared gradients, and Adam combines both ideas. The suitable optimizer depends on the model and dataset.